

Assignment 1: Word Analogy

NTHU Natural Language Processing

Hung-Yu Kao

IKM Lab TAs



What is analogy?

- Anthology helps to assess how well the word embedding model captures semantic and syntactic relationships between words.

The analogy task is often summarized by the famous phrase:

A is to **B** as **C** is to **D**. (e.g. **King** is to **Queen** as **Man** is to **Woman**.)



In euqation: $\overrightarrow{\text{King}} + \overrightarrow{\text{Queen}} - \overrightarrow{\text{Man}} \approx \overrightarrow{\text{Woman}}$

Dataset

Google Word Analogy (Mikolov et al., 2013)

- 19,544 entries
 - 5 sub-categories for **semantic** relationships
 - 9 sub-categories for **syntactic** relationships
- Each entry: four words, separated by a space. (e.g. **A B C D**)
- Task: **A** is to **B** in the same sense that **C** is to ?
 - The forth word (**D**) in each entry **is the answer**.

Dataset examples

*Answer in red

- Task: **A** is to **B** in the same sense that **C** is to ?

Category	Sub-category	Example
Semantic	family	king queen man woman
Syntactic	gram1-adjective-to-adverb	infrequent infrequently cheerful cheerfully

Code Template

Coding environment

<https://colab.research.google.com/>

Google colab

[Blog](#)

[Release Notes](#)

[Notebooks](#)

[Resources](#)

[Open Colab](#)

[New Notebook](#)

[Sign Up](#)



Google Colaboratory

Colab is a hosted Jupyter Notebook service that requires no setup to use and provides free access to computing resources, including GPUs and TPUs. Colab is especially well suited to machine learning, data science, and education.

[Open Colab](#)

[New Notebook](#)



Brief Introduction to Colab

- Cloud-Based Environment: Colab is entirely online, so no installation is required. You can write and run Python code directly in your browser.
- **Free Access to Hardware:** It provides access to **GPUs** and **TPUs for free**, making it ideal for machine learning, data science, and AI tasks.
- **Easy to use:** Colab uses **Jupyter notebooks**, a widely used format that allows combining code, output, and markdown text in a single document.

Quick view of the Colab editor

The screenshot displays the Google Colab editor interface. At the top, the 'main.ipynb' tab is active, with a star icon for bookmarks. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help', with a status message 'All changes saved'. On the right, there are links for 'Comment', 'Share', and a settings gear, along with RAM and Disk usage indicators and a 'Gemini' icon.

The interface is divided into three main sections:

- File region:** Located on the left, it contains a 'Files' sidebar with icons for file operations (upload, refresh, download, delete) and a list of files: '..', 'sample_data', and 'questions-words...'. A red box highlights this area with the label 'File region'.
- Text block:** A red box at the top of the main editor area contains the text 'Text block (You can write your notes with Markdown.)' and a dropdown menu with the option 'Part I: Data Pre-processing'.
- Code blocks (with outputs):** The main editor area contains a code block with the following content:

```
[1] import pandas as pd
```

Download the Google Analogy dataset
!wget http://download.tensorflow.org/data/questions-words.txt

--2024-09-13 07:54:37-- http://download.tensorflow.org/data/questions-words.txt
Resolving download.tensorflow.org (download.tensorflow.org)... 172.217.164.27, 172.217.0.91, 172.217.12.27, ...
Connecting to download.tensorflow.org (download.tensorflow.org)|172.217.164.27|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 603955 (590K) [text/plain]
Saving to: 'questions-words.txt'

questions-words.txt 100%[=====] 589.80K --.-KB/s in 0.002s

2024-09-13 07:54:37 (242 MB/s) - 'questions-words.txt' saved [603955/603955]

A red box highlights this code block and its output with the label 'Code blocks (with outputs)'.

Assignment 1 materials

GitHub: [LINK](#)

The screenshot shows a GitHub repository interface. The left sidebar displays the file tree with the following structure:

- Files
 - main
 - Go to file
 - 2024
 - 2025
 - 2026
 - Assignments
 - Assignment1
 - NLP_HW1_word_emb.pdf
 - README.md
 - main.ipynb
 - questions-words.csv
 - README.md
 - Reference
 - Slides

The main content area shows the commit history for the 'Assignment1' directory. The commit message is 'Add 2026 Assignment1' by user 'FengZhiQuan1046' at 'cd41777' 19 minutes ago. The commit history table is as follows:

Name	Last commit message	Last commit date
..		
NLP_HW1_word_emb.pdf	Add 2026 Assignment1	19 minutes ago
README.md	Add 2026 Assignment1	19 minutes ago
main.ipynb	Add 2026 Assignment1	19 minutes ago
questions-words.csv	Add 2026 Assignment1	19 minutes ago

Below the table, the 'README.md' file is open, showing the text 'Assignment 1'.

Download dataset

▼ Part I: Data Pre-processing

✓
1 秒

```
[1] import pandas as pd
```

✓
0 秒



```
# Download the Google Analogy dataset  
!wget http://download.tensorflow.org/data/questions-words.txt
```

→ The downloaded data is a **text file**.

Check the downloaded file

- You can first browse the file on Colab!

Line-by-line manner

questions-words.txt ×

```
1 : capital-common-countries
2 Athens Greece Baghdad Iraq
3 Athens Greece Bangkok Thailand
4 Athens Greece Beijing China
5 Athens Greece Berlin Germany
6 Athens Greece Bern Switzerland
7 Athens Greece Cairo Egypt
8 Athens Greece Canberra Australia
9 Athens Greece Hanoi Vietnam
10 Athens Greece Havana Cuba
11 Athens Greece Helsinki Finland
12 Athens Greece Islamabad Pakistan
13 Athens Greece Kabul Afghanistan
14 Athens Greece London England
15 Athens Greece Madrid Spain
16 Athens Greece Moscow Russia
17 Athens Greece Oslo Norway
18 Athens Greece Ottawa Canada
19 Athens Greece Paris France
20 Athens Greece Rome Italy
```

Load data and check the first ten entries

```
✓  
0 秒 [3] # Preprocess the dataset  
      file_name = "questions-words"  
      with open(f"{file_name}.txt", "r") as f:  
          data = f.read().splitlines()
```

```
✓  
0 秒 [4] # check data from the first 10 entries  
      for entry in data[:10]:  
          print(entry)
```

```
⇒ : capital-common-countries  
   Athens Greece Baghdad Iraq  
   Athens Greece Bangkok Thailand  
   Athens Greece Beijing China  
   Athens Greece Berlin Germany  
   Athens Greece Bern Switzerland  
   Athens Greece Cairo Egypt  
   Athens Greece Canberra Australia  
   Athens Greece Hanoi Vietnam  
   Athens Greece Havana Cuba
```

← Sub-category

TODOs

#TODO1	Convert the analogy data to pd.DataFrame (evaluation dataset) (5%)
#TODO2	Get predictions of the analogy task using pre-trained word embeddings (10%)
#TODO3	Plot t-SNE to see word relationships in the sub-category of `family` (5%)
#TODO4	Sample 20% Wikipedia articles (5%)
#TODO5	Train your own word embeddings with the sampled articles (10%)
#TODO6	Get predictions of the analogy task using your trained word embeddings (10%)
#TODO7	Plot t-SNE to see word relationships in the sub-category of `family` (10%)



1. Use pre-trained word embeddings



2. Train your own word embeddings

TODO1: Convert data to pd.DataFrame (5%)

```
[ ] # TODO1: Write your code here for processing data to pd.DataFrame  
# Please note that the first five mentions of ": " indicate `semantic`,  
# and the remaining nine belong to the `syntactic` category.
```

```
⤵ : capital-common-countries  
Athens Greece Baghdad Iraq  
Athens Greece Bangkok Thailand  
Athens Greece Beijing China  
Athens Greece Berlin Germany  
Athens Greece Bern Switzerland
```



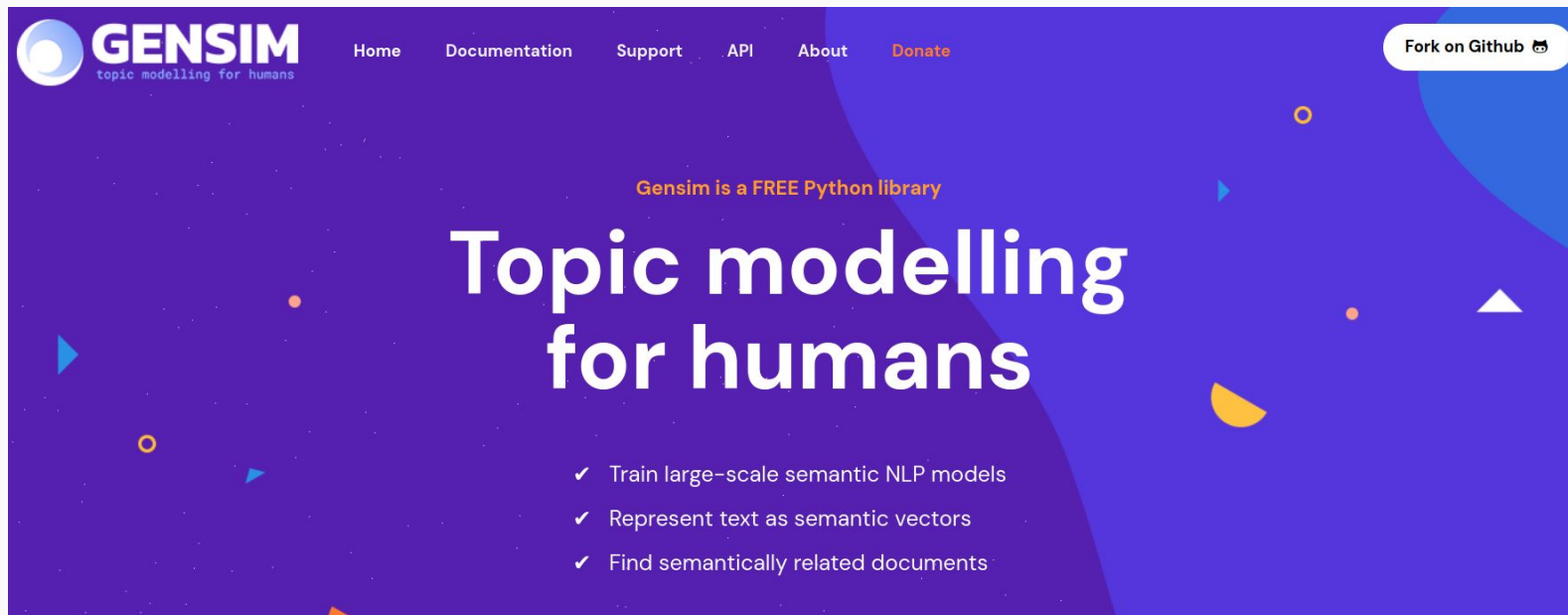
✓ 0 秒 df.head()



	Question	Category	SubCategory
0	Athens Greece Baghdad Iraq	Semantic	: capital-common-countries
1	Athens Greece Bangkok Thailand	Semantic	: capital-common-countries
2	Athens Greece Beijing China	Semantic	: capital-common-countries
3	Athens Greece Berlin Germany	Semantic	: capital-common-countries
4	Athens Greece Bern Switzerland	Semantic	: capital-common-countries



Gensim: A popular Python package for embeddings



<https://radimrehurek.com/gensim/index.html>

Load the word embedding model with Gensim

You can also try other models!

✓
37
秒



```
MODEL_NAME = "glove-wiki-gigaword-100"  
data = pd.read_csv("questions-words.csv")
```

```
# Load the pre-trained model (using GloVe vectors here)  
model = gensim.downloader.load(MODEL_NAME)  
print("The Gensim model loaded successfully!")
```



The Gensim model loaded successfully!

TODO2: Get predictions of the analogy task using pre-trained word embeddings (10%)

```
[ ] # Do predictions and preserve the gold answers (word_D)
preds = []
golds = []

for analogy in tqdm(data["Question"]):
    # TODO2: Write your code here to use pre-trained word embeddings for getting predictions of the analogy task.
    # You should also preserve the gold answers during iterations for evaluations later.
    """ Hints
    # Unpack the analogy (e.g., "man", "woman", "king", "queen")
    # Perform vector arithmetic: word_b + word_c - word_a should be close to word_d
    # Source: https://github.com/piskvorky/gensim/blob/develop/gensim/models/keyedvectors.py#L776
    # Mikolov et al., 2013: big - biggest and small - smallest
    # Mikolov et al., 2013: X = vector("biggest") - vector("big") + vector("small").
    """
```

Do evaluations (We've finished for you to test!)

```
[ ] # Perform evaluations. Do not modify this block!!

def calculate_accuracy(gold: np.ndarray, pred: np.ndarray) -> float:
    return np.mean(gold == pred)

golds_np, preds_np = np.array(golds), np.array(preds)
data = pd.read_csv("questions-words.csv")

# Evaluation: categories
for category in data["Category"].unique():
    mask = data["Category"] == category
    golds_cat, preds_cat = golds_np[mask], preds_np[mask]
    acc_cat = calculate_accuracy(golds_cat, preds_cat)
    print(f"Category: {category}, Accuracy: {acc_cat * 100}%")

# Evaluation: sub-categories
for sub_category in data["SubCategory"].unique():
    mask = data["SubCategory"] == sub_category
    golds_subcat, preds_subcat = golds_np[mask], preds_np[mask]
    acc_subcat = calculate_accuracy(golds_subcat, preds_subcat)
    print(f"Sub-Category{sub_category}, Accuracy: {acc_subcat * 100}%")
```

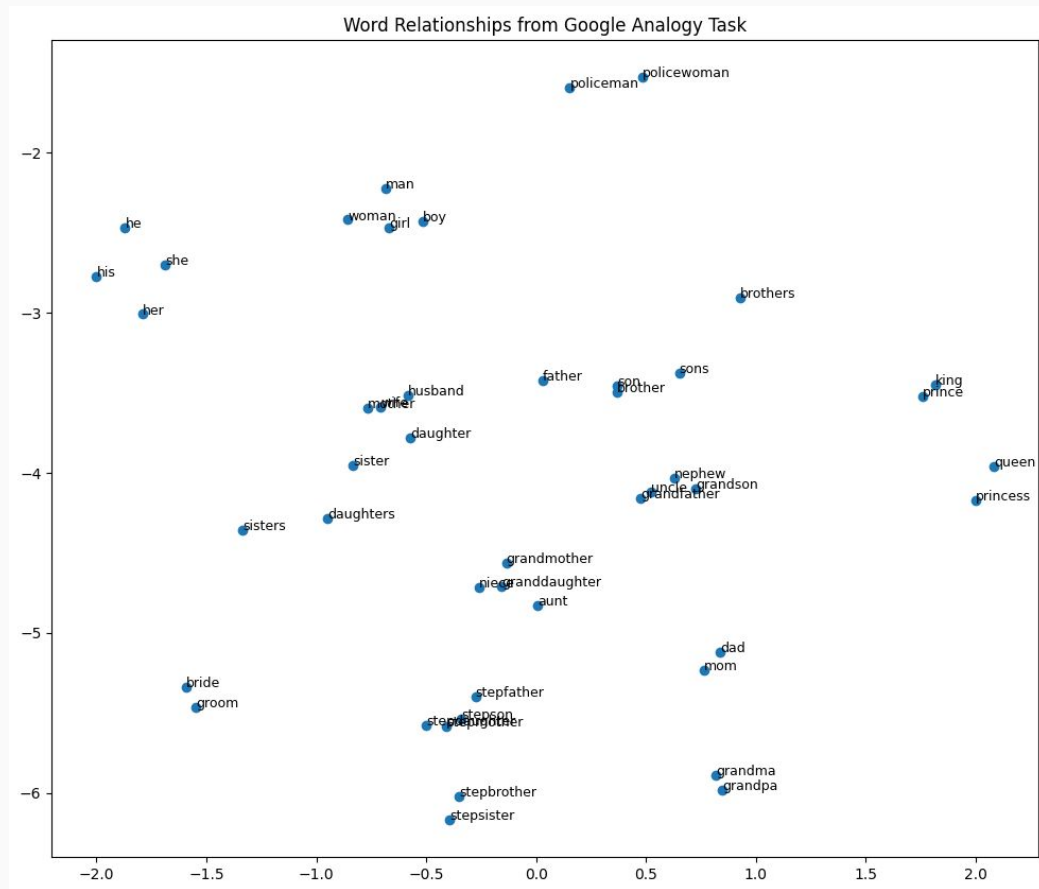
We will provide this file to you. But you still need to write code for #TODO1.

TODO3: Plot t-SNE to see word relationships in the sub-category of `family` (5%)

```
[ ] # Collect words from Google Analogy dataset
    SUB_CATEGORY = ": family"

    # TODO3: Plot t-SNE for the words in the SUB_CATEGORY `: family`

    plt.title("Word Relationships from Google Analogy Task")
    plt.show()
    plt.savefig("word_relationships.png", bbox_inches="tight")
```



Train your own word embeddings

wikidatawiki dump progress on 20240901

Currently 6M+
articles for EN Wiki

This is the Wikimedia dump service. Please read the [copyrights](#) information. See [Meta:Data dumps](#) for documentation on the provided data formats.

Older versions of the 7zip decoder on Windows are known to have [problems](#) with some bz2-format files for larger wikis; we recommend the use of bzip2 for Windows for these cases. 7zip 20.00 alpha (x64) on Win10 LTSC 2019 and later versions should work properly.

Please report problems with these dumps on Phabricator and add the [Dumps-generation](#) tag.

See [all databases list](#).

This dump is in progress; see also the [previous dump from 2024-08-20](#)

For a machine-readable version of the information on this page, see [the json status file](#).

An example of raw data of Wiki dump in XML

```
<page>
  <title>Anarchism</title>
  <ns>0</ns>
  <id>12</id>
  <revision>
    <id>1243078189</id>
    <parentid>1243073506</parentid>
    <timestamp>2024-08-30T11:06:56Z</timestamp>
```

"Anarchism" is a [[political philosophy]] and [[Political movement|movement]] that is against all forms of [[authority]] and seeks to abolish the [[institutions]] it claims maintain unnecessary [[coercion]] and [[Social hierarchy|hierarchy]], typically including the [[state (polity)|state]] and [[capitalism]]. Anarchism advocates for the replacement of the state with [[Stateless society|stateless societies]] and [[Voluntary association|voluntary]] [[Free association (communism and anarchism)|free associations]]. As a historically [[left-wing]] movement, this reading of anarchism is placed on the [[Far-left politics|farthest left]] of the [[political spectrum]], usually described as the [[libertarian]] wing of the [[socialist movement]] ([[libertarian socialism]]).

Wiki corpus pre-processing with Gensim

```
class gensim.corpora.wikicorpus.WikiCorpus(fname, processes=None, lemmatize=None, dictionary=None, metadata=False, filter_namespaces=('0', ), tokenizer_func=<function tokenize>, article_min_tokens=50, token_min_len=2, token_max_len=15, lower=True, filter_articles=None)
```

Bases: `TextCorpus`

Treat a Wikipedia articles dump as a read-only, streamed, memory-efficient corpus.

Supported dump formats:

- `<LANG>wiki-<YYYYMMDD>-pages-articles.xml.bz2`
- `<LANG>wiki-latest-pages-articles.xml.bz2`

The documents are extracted on-the-fly, so that the whole (massive) dump can stay compressed on disk.

Notes

Dumps for the English Wikipedia can be founded at <https://dumps.wikimedia.org/enwiki/>.

We've done Wiki data cleaning for you!

- Downloading the raw Wiki file takes time.
- Cleaning the raw Wiki corpus takes even much longer time.

```
[ ] # Download the split Wikipedia files
    # Each file contain 562365 lines (articles).
    !gdown --id 1E3LV7I6Wruqqwf4dmU_n7nRCtniXX7p2 -O wiki_texts_part_0.txt.gz
    !gdown --id 1h1fnzSXGmAW0I2K11Nm1a1CBK80yCbc -O wiki_texts_part_1.txt.gz
```

Examples! There will be 11 files in total.

```
[ ] # Extract the downloaded wiki_texts_parts files.
    !gunzip -k wiki_texts_part_*.gz
```

If this is not available, you can also download through this [LINK](#).

TODO4: Sample 20% Wikipedia articles (5%)

```
[ ] # Combine the extracted wiki_texts_parts files.  
    !cat wiki_texts_part_*.txt > wiki_texts_combined.txt
```

```
[ ] # Check the first ten lines of the combined file  
    !head -n 10 wiki_texts_combined.txt
```

```
[ ] # Now you need to do sampling because the corpus is too big.  
    # You can further perform analysis with a greater sampling ratio.  
  
import random  
  
wiki_txt_path = "wiki_texts_combined.txt"  
# wiki_texts_combined.txt is a text file separated by linebreaks (\n).  
# Each row in wiki_texts_combined.txt indicates a Wikipedia article.  
  
with open(wiki_txt_path, "r", encoding="utf-8") as f:  
    with open(output_path, "w", encoding="utf-8") as output_file:  
        # TODO4: Sample `20%` Wikipedia articles  
        # Write your code here
```

TODO5: Train your own word embeddings with the sampled articles (10%)

```
[ ] # TODO5: Train your own word embeddings with the sampled articles
# https://radimrehurek.com/gensim/models/word2vec.html#gensim.models.word2vec.Word2Vec
# Hint: You should perform some pre-processing before training.
```

Examples

Initialize and train a `Word2Vec` model

```
>>> from gensim.models import Word2Vec
>>> sentences = [["cat", "say", "meow"], ["dog", "say", "woof"]]
>>> model = Word2Vec(sentences, min_count=1)
```

Recommended data pre-processing steps

- Remove non-English words
- Remove stop words
- Lemmatization (e.g., rocks -> rock)
- Better tokenization than simply splitting words based on whitespaces
- Retain the most frequent words for the dictionary

*Note that not every tricks yield better performance. You should try on your own.

TODO6: Get predictions of the analogy task using your trained word embeddings (10%)

Same as #TODO2, you still need to get predictions for your trained embeddings.

```
[ ] # Do predictions and preserve the gold answers (word_D)
    preds = []
    golds = []

    for analogy in tqdm(data["Question"]):
        # TODO6: Write your code here to use your trained word embeddings for getting predictions of the analogy task.
        # You should also preserve the gold answers during iterations for evaluations later.
        """ Hints
        # Unpack the analogy (e.g., "man", "woman", "king", "queen")
        # Perform vector arithmetic: word_b + word_c - word_a should be close to word_d
        # Source: https://github.com/piskvorky/gensim/blob/develop/gensim/models/keyedvectors.py#L776
        # Mikolov et al., 2013: big - biggest and small - smallest
        # Mikolov et al., 2013: X = vector("biggest") - vector("big") + vector("small").
        """
```

TODO7: Plot t-SNE to see word relationships in the sub-category of `family` (10%)

Same as #TODO3, you still need to plot t-SNE for your trained embeddings.

```
[ ] # Collect words from Google Analogy dataset
    SUB_CATEGORY = ": family"

    # TODO7: Plot t-SNE for the words in the SUB_CATEGORY `: family`

    plt.title("Word Relationships from Google Analogy Task")
    plt.show()
    plt.savefig("word_relationships.png", bbox_inches="tight")
```

Scoring

Coding work : **55%** (the seven TODOs, **please follow our code template**)

The .ipynb file must include both your code and its execution outputs (100% version of wiki). If the execution results are not included, no points can be awarded for any parts related to experimental results or plots.

Report: **45%** (**please write it in your submitted .ipynb**)

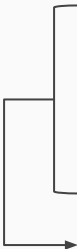
- Which embedding model do you use? What are the pre-processing steps? What are the hyperparameter settings? **(5%)**
- What will the performance be like if you sample 5%, 10% and 20% of wiki text in TODO4? **(10%, 3% for each)**
- What is the performance for different categories or sub-categories when trained on different corpora (except wiki)? **(15%) You should include your additional code in the notebook below the report block; otherwise, you will receive 0 points.**
 - Present your results. **(5%)**
 - Introduce the corpus you selected and explain what are the differences between the Wikipedia corpus and your corpus. (including data size, topic difference, structural difference ...) **(5%)**
 - Explain why the performance increases or decreases. **(5%)**
- Select at least five words and use their embeddings to retrieve the five most similar words for each word you choose. What do you observe? **(10%)**
- **... Anything that can strengthen your report. (5%)**

Delivery policies: File formats

- Coding work: Python notebook file (.ipynb)
 - Download your script via Colab.
- Package list: requirements.txt
 - E.g., gensim==4.3.3
- **No other formats are allowed.**
- Zip the files above before uploading your assignment.



Delivery policies: Filenames



	Filename rule	Filename example
Coding work	NLP_HW1_ school _student_ID.py	NLP_HW1_ NTHU _12345678.py
Package list	requirements.txt	
Zipped file	NLP_HW1_ school _student_ID.zip	NLP_HW1_ NTHU _12345678.zip

- You do not need to upload the code for your additional experiments; only the training on the provided WikiText template is required.

Delivery policies: Things You should include

- In your report:

	Example	
Environment types	If Colab or Kaggle	If local
Running environment	Colab	System: Ubuntu 22.04, CPU: Ryzen 7-7800X3D
Python version	Colab	Python 3.10.1

Delivery policies: Rules of coding

- If you use Generative AI, please specify your usage **both** in:
 - **Code comments**
 - **Reports**
- **No plagiarism.** You should not copy and paste from your classmates.
- Please provide links if you take the code from the Internet as reference.
- The following behaviors **will cause loss in the score of the assignment**: (1) **Usage with Generative AI without specifications** (2) **Internet sources without specifications** (3) **Plagiarism.**

Punishments

Rule	Name your code: NLP_HW1_ school _st udent_ID.py (only .py is acceptable)	Name your file: NLP_HW1_ school _stu dent_ID.zip	Include requirements.txt
Punishment	-5	-5	-5
Rule	Include python version in your report	Do not modify the code template (only changes to data loading are allowed).	Your code or report should not shows a high degree of similarity to another student's submission.
Punishment	-5	-5	-100 for both

If you are using Colab, go to **File** → **Download** → **Download .py** to obtain the Python file.

Uploading the zipped file

- Please upload your file to NTU COOL.
- You will have three weeks to finish this assignment.